BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure,
Environment,
Actuators,
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width, self.height, self.walls, self.food_positions, self.agent_pos, self.opponents

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

autonomous entities that operate alongside the primary agent.
Active Decision-Making: During each execution step, these opponents actively take their own independent actions by randomly selecting a movement direction.
Agent Interaction: The actions of the opponents directly impact the primary agent's

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The absolute, cumulative historical log of every single percept the agent has received from its initialization up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The get_percept() method represents the Sensors component of the PEAS framework.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable.
In a fully observable environment, an agent's sensors give it access to the complete state of the environment at any given time. However, the get_percept() dictionaries

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

When execute_action() deducts points for hitting a wall, the Performance Measure component of the PEAS framework is being actively updated.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important that the performance measure evaluates the external environment state rather than internal processing effort because it aligns the agent's incentives with the actual goals of the system.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__` . Populate it with coordinate tuples safely avoiding position `(0, 0)` , walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

If you add self.toxic_traps to the environment but intentionally omit this data from the get_percept() dictionary, the environment remains decisively classified as Partially Observable.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

By introducing the 'smells_toxin' sensor into the get_percept() dictionary, you directly enhance the agent's ability to act rationally. In artificial intelligence, a rational agent is one that selects actions to maximize its expected utility (which, in

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

In this scenario, if the human designer creates a performance measure that rewards the vacuum agent for the action of cleaning up dirt (e.g., +10 points for every piece of dirt vacuumed), rather than the state of having a clean floor, a rational agent will

Finalize and Lock Lab 01 Analytical Evaluation

FACULTY OF COMPUTING